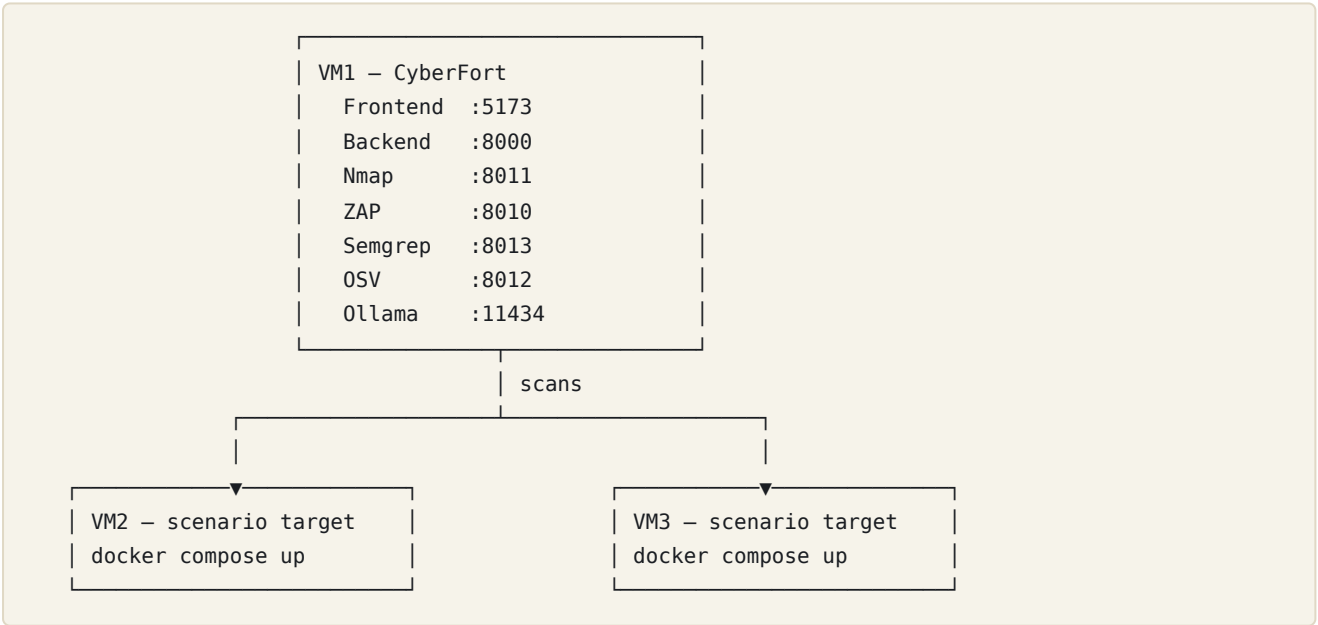


CyberFort cyber-range training scenarios

A set of three deliberately-vulnerable training exercises that showcase the **CyberFort** compliance platform on a three-VM cyber range. Each scenario is from a **different sector** and is assessed against a **different compliance framework**, so the three together demonstrate the full breadth of the platform.

#	Folder	Sector	Compliance framework	Target stack	Primary CyberFort capabilities
1	01-portpilot-maritime/	Maritime	CRA (Cyber Resilience Act, Annex I + Vuln. Handling)	Flask + PostgreSQL	Nmap + ZAP + Semgrep + OSV + CRA conformity assessment + AI assistant
2	02-smartgrid-meter-energy/	Energy	NIS2 (Article 21(2) measures)	Flask admin + Mosquitto MQTT + Modbus TCP	Nmap (OT ports) + ZAP + Semgrep + OSV + NIS2 assessment
3	03-netlink-isp-digital-infra/	Digital infrastructure	ISO 27001:2022 (Annex A gap analysis)	Node.js Express + PostgreSQL	Nmap + ZAP + Semgrep (Node.js) + OSV (npm) + ISO 27001 assessment + JWT-forgery chain

Range topology



All three VMs sit on the same isolated training subnet. CyberFort reaches each scenario by IP. Scenarios run on either VM2 or VM3 depending on the day's session plan; only one scenario should be active per target VM at a time because their port mappings overlap (Postgres on 5432 in S1+S3).

What the three scenarios show together

- **Three sectors** — maritime port operator, energy distribution operator, ISP. These mirror the SME profiles the CYBERFORT proposal explicitly targets.
- **Three frameworks** — CRA (product-side regulatory compliance), NIS2 (operator-side essential-entity obligations), ISO 27001 (general ISMS gap analysis). Each one is seeded in CyberFort with question texts the trainee actually answers.
- **Three technology stacks** — Python + Flask, Python + OT protocols (Modbus + MQTT), Node.js + EJS. Different scanner footprints (Semgrep covers Python and Node, OSV covers PyPI and npm).
- **One workflow** — register product → run scanners → convert findings to risks → answer the assessment → attach evidence → export PDF readiness report.

How to use the scenarios

For each scenario the intended flow is:

1. Cyber range admin deploys the scenario stack on a target VM (`docker compose up -d --build`).
2. Trainee logs into CyberFort on VM1.
3. Trainee follows `docs/TRAINEE_HANDBOOK.md` for that scenario — register the target as a product, run the relevant scanners, fill in the assessment, export the PDF readiness report.
4. Instructor reviews the PDF and the risk-register entries against `docs/INSTRUCTOR_GUIDE.md` .

Each scenario is sized for **two hours** of trainee time at intermediate level.

Repo layout

```
cyber-range-scenarios/
├─ README.md                ← this index
├─ scripts/
│   └─ md_to_pdf.py         ← markdown → PDF helper used to render the docs
├─ 01-portpilot-maritime/   ← Maritime / CRA / Flask
├─ 02-smartgrid-meter-energy/ ← Energy / NIS2 / Flask + MQTT + Modbus
└─ 03-netlink-isp-digital-infra/ ← Digital infra / ISO 27001 / Node.js
```

Every scenario folder follows the same structure: `README.md` , `docker-compose.yml` , application source, `docs/` (handbook + instructor guide + control mapping). PDFs of all the markdown docs ship next to their `.md` counterparts and are regenerated via `python3 scripts/md_to_pdf.py <file>.md` .

Safety

Each scenario is intentionally vulnerable. **Run only on an isolated cyber-range network** — never on the public internet or on a corporate LAN.

Status

- ✓ Scenario 01 (PortPilot Maritime, CRA): complete, smoke-tested, PDFs rendered.
- ✓ Scenario 02 (SmartGrid Meter Energy, NIS2): complete, smoke-tested, PDFs rendered.

- ✓ Scenario 03 (NetLink ISP Digital Infrastructure, ISO 27001): complete, smoke-tested, PDFs rendered.